



Laboratory Report #5
Predictive Modeling Using Scikit-learn and Pandas

Name:	Lyka Sophia S. Jardeleza	Date:	05/25/2025
Program:	BSCpE	Section:	A361

A. Instructions:

Commented [JC1]: Always edit the # and the place the complete laboratory exercise title.

A. Load and Explore the Dataset

```
Step 1: Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error

Step 2: Load dataset
df = pd.read_csv('student_performance.csv')
print(df.head())

Step 3: Basic EDA
print(df.describe())
print(df.isnull().sum())
df.plot(kind='scatter', x='StudyHours', y='Score')
plt.show()
```

B. Predicting Exam Scores (Linear Regression)

```
Step 4: Define features and target
X = df[['StudyHours', 'AttendanceRate']]
y = df['Score']

Step 5: Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Step 6: Train and predict
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print('MSE:', mean_squared_error(y_test, predictions))
print("Linear Regression Equation: Score = {:.2f} * StudyHours + {:.2f} * AttendanceRate + {:.2f}".format(
    lin_model.coef_[0], lin_model.coef_[1], lin_model.intercept_
))
```

C. Predicting Passing Status (Logistic Regression)

```
Step 7: Define new target
y_class = df['Passed']

Step 8: Train/test split and model training
X_train, X_test, y_train, y_test = train_test_split(X, y_class, test_size=0.2, random_state=42)
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
preds = log_model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, preds))
print("Logistic Regression Coefficients:")
print(f" StudyHours: {log_model.coef_[0][0]:.4f}")
print(f" AttendanceRate: {log_model.coef_[0][1]:.4f}")
print(f" Intercept: {log_model.intercept_[0]:.4f}")
```



Laboratory Report #5
Predictive Modeling Using Scikit-learn and Pandas

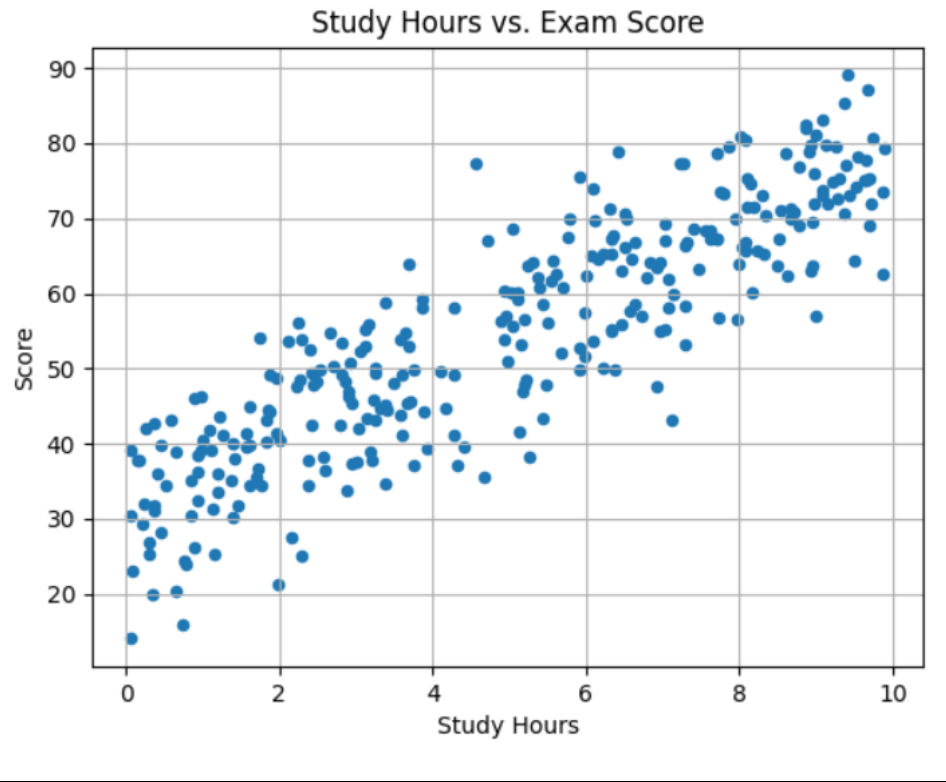
Screenshot of Output

```
"C:\Users\Lyka\Sophia\PycharmProjects\PythonProject\venv\Scripts\python.exe" "C:\Users\Lyka\Sophia\PycharmProjects\PythonProject\JARDELEZA_05100L_Lab5.py"
First 5 rows of dataset:
  StudyHours  AttendanceRate  Score  Passed
0         3.75             52.58  37.13      0
1         9.51             76.57  74.21      1
2         7.32             77.03  66.88      1
3         5.99             81.87  57.52      0
4         1.56             86.30  39.55      0

Dataset Description:
  StudyHours  AttendanceRate  Score  Passed
count  300.000000      300.000000  300.000000  300.000000
mean     4.952033       75.541733  54.599967  0.403333
std     2.943406       15.129449  16.802330  0.491386
min     0.050000       50.540000  14.050000  0.000000
25%     2.395000       62.480000  42.075000  0.000000
50%     5.115000       76.085000  54.910000  0.000000
75%     7.572500       88.670000  67.190000  1.000000
max     9.900000       99.990000  89.050000  1.000000

Missing Values:
StudyHours      0
AttendanceRate  0
Score           0
Passed          0
dtype: int64

Mean Squared Error (MSE): 30.84642628201685
Linear Regression Equation: Score = 4.85 * StudyHours + 0.36 * AttendanceRate + 3.57
Classification Accuracy: 0.9333333333333333
Logistic Regression Coefficients:
StudyHours: 1.7905
AttendanceRate: 0.1309
Intercept: -20.6055
```





Laboratory Report #5

Predictive Modeling Using Scikit-learn and Pandas

Source Code

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error

# A. Load and Explore the Dataset
# Step 2: Load dataset
df = pd.read_csv('student_performance.csv')
print("First 5 rows of dataset:")
print(df.head())

# Step 3: Basic EDA
print("\nDataset Description:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

df.plot(kind='scatter', x='StudyHours', y='Score')
plt.title("Study Hours vs. Exam Score")
plt.xlabel("Study Hours")
plt.ylabel("Score")
plt.grid(True)
plt.show()

# B. Predicting Exam Scores (Linear Regression)
# Step 4: Define features and target
X = df[['StudyHours', 'AttendanceRate']]
y = df['Score']

# Step 5: Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 6: Train and predict
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)

predictions = lin_model.predict(X_test)

print("\nMean Squared Error (MSE):", mean_squared_error(y_test, predictions))
print("Linear Regression Equation: Score = {:.2f} * StudyHours + {:.2f} * AttendanceRate + {:.2f}".format(
    lin_model.coef_[0], lin_model.coef_[1], lin_model.intercept_
))

# C. Predicting Passing Status (Logistic Regression)
# Step 7: Define new target
y_class = df['Passed']

# Step 8: Train/test split and model training
X_train, X_test, y_train, y_test = train_test_split(X, y_class, test_size=0.2, random_state=42)

log_model = LogisticRegression()
log_model.fit(X_train, y_train)

preds = log_model.predict(X_test)

print("\nClassification Accuracy:", accuracy_score(y_test, preds))
print("Logistic Regression Coefficients:")
print(f" StudyHours: {log_model.coef_[0][0]:.4f}")
print(f" AttendanceRate: {log_model.coef_[0][1]:.4f}")
print(f" Intercept: {log_model.intercept_[0]:.4f}")
```



Laboratory Report #5
Predictive Modeling Using Scikit-learn and Pandas